Safe Fruit:
Exact Branch-and-Bound with Forest DP

Course: Data Structures and Algorithms

Project: Safe Fruit

Implementation: C99

Completion Date: October 31, 2026

You Longbin 3250106392

Huang Yirui 3250104426

Chen Zhikun 3250100883

Generated with LaTeX

Contents

1 Introduction **3**
- 1.1 Description . . . 3
- 1.2 Overview of the Implementation . . . 4

2 Algorithm Specification **5**
- 2.1 Encoding the Two Objectives . . . 5
- 2.2 Branch and Bound . . . 5
- 2.3 Matching Upper Bound . . . 6
- 2.4 Component Decomposition . . . 6
- 2.5 Tree DP . . . 6
- 2.6 Residual Forest DP . . . 7
- 2.7 When to Check: Versions 7 and 8 . . . 7
- 2.8 Final Algorithm . . . 7
- 2.9 Correctness . . . 8
- 2.10 Complexity . . . 8

3 Testing Results **9**
- 3.1 Test Cases and Input Data . . . 9
- 3.2 Measurement Method . . . 9
- 3.3 Sample Run . . . 10
- 3.4 Additional Automated Tests . . . 10
- 3.5 Performance by Optimization . . . 10

4 Analysis and Comments **13**
- 4.1 Why the Algorithm Is Exact . . . 13
- 4.2 Why the Optimizations Matter . . . 13
- 4.3 PTA-Specific Pitfalls . . . 13
- 4.4 Limitations . . . 14
- 4.5 Lessons Learned . . . 14
- 4.6 Summary . . . 14

References **15**

A Source Location **16**

Chapter 1. Introduction

1.1 Description

The input describes a basket of fruits and a list of warnings. Each warning forbids one pair of fruits from being eaten together. We must choose a subset of the basket such that no warned pair is chosen. Among all such subsets, the program must first maximize the number of chosen fruits and then minimize their total price. The problem guarantees that the optimal answer is unique.

The input format is:

- the first line contains N warnings and M basket fruits;
- the next N lines each contain a forbidden pair of fruit IDs;
- the next M lines each contain a fruit ID and its price.

Here $N, M \leq 100$, every price is between 1 and 1000, and fruit IDs are printed with three digits. The output has exactly three lines: the number of chosen fruits, their IDs in increasing order separated by exactly one space, and the total price. There must be no trailing space on the ID line.

Let each basket fruit be a vertex and each warning between two basket fruits be an undirected edge. A feasible choice is an independent set of this graph. The task is therefore a *lexicographic maximum independent set* problem: maximize the cardinality first, then minimize the sum of vertex prices.

The lexicographic order is important. A greedy rule that picks the cheapest fruit first can fail. For example, on the path $1 - 2 - 3$ with prices $p_1 = 100$, $p_2 = 1$, $p_3 = 100$, the cheapest fruit is 2, but choosing it gives only one fruit; the optimal choice is $\{1, 3\}$ with two fruits and total price 200. The program must therefore optimize the count before the price, not the price alone.

General maximum independent set is NP-hard, so no polynomial-time exact algorithm is expected. The assignment limits M to 100, which makes an exact branch-and-bound search with strong pruning practical.

A warning may mention a fruit that is not in the basket. Such a warning does not constrain the selection, and the program ignores it defensively. The PTA sample does not actually contain one: its first 16 lines are warnings, and the lines `020 99` and `019 99` that follow are fruit lines, meaning that fruits 020 and 019 each cost 99. The special tests in Chapter 3 include outside warnings explicitly.

The sample has 16 warnings and 20 fruits. The warnings form three nontrivial components: the edge $001 - 002$; the 6-cycle $003 - 004 - 005 - 006 - 007 - 008 - 003$; and the tree on 009–018, where 009 is adjacent to 010, 011, 012, 013, and 010-014, 011-015, 012-016, 012-017, 013-018 are the remaining edges. Fruits 019 and 020 have no warnings and cost 99 each. The sample answer is

```
002 004 006 008 009 014 015 016 017 018 019 020
```

with total price 239. The choices 002, 004, 006, and 008 come from the first two components; 009 and 014–018 come from the tree; 019 and 020 are isolated and are selected because they increase the count. The price check is 1+4+5+1+10+5+6+3+2+4+99+99 =

239. This is a useful sanity check for the score encoding and for the isolated-fruit rule.

The final program should be exact, implement the count-first, price-second objective, print the required format, and run comfortably for $M \leq 100$. The submitted program is `v8_cycle_tree_dp.c`. Earlier versions are kept in the repository because they were used to validate each optimization during development; they are not alternative answers.

1.2 Overview of the Implementation

The program is organized around four components.

1. **Graph construction and PTA input.** The program reads N and M, maps the three-digit IDs to internal indices, and builds both a bitset adjacency and adjacency lists. Isolated fruits are added to the answer directly. A warning with an endpoint outside the basket is ignored; this is defensive, because the sample has no such warning.
2. **Score encoding and search.** Each fruit receives the weight $B - p_v$, so one integer captures both objectives. The search uses two 64-bit masks for the candidate and selected sets, a matching upper bound, and a dynamic maximum-degree pivot.
3. **Structural decomposition.** Original connected components are solved independently. A tree component uses tree DP; a cyclic component uses branch and bound, with residual-forest DP when the candidate graph becomes acyclic. Version 8 chooses when to perform the forest check.
4. **Output.** The selected IDs are sorted in increasing order and printed with `%03d`; a space is printed before every ID except the first, so the line has no trailing space. The second line is followed by the total price on the third line.

The count on the first line is the number of selected IDs, and the price is their sum. The input is assumed to be a simple conflict graph, as the assignment states that there are no duplicated tips; self-loops are not specified.

The final program is compiled with `gcc -O2 -std=c99`; the optional `-DBENCH` macro is used only for local timing and is not needed by the judge. Chapter 2 specifies the algorithm and its correctness; Chapter 3 is the test report; Chapter 4 gives the analysis and comments; the appendix gives the source location.

Chapter 2. Algorithm Specification

2.1 Encoding the Two Objectives

Let p_v be the price of fruit v. Define

$$B = 100001, \qquad w_v = B - p_v, \qquad W(S) = B|S| - \sum_{v \in S} p_v.$$

The maximum possible total price is $100 \times 1000 = 100000 < B$, and every w_v is positive.

Lemma 1. *For any two feasible sets A and C, $W(A) > W(C)$ if and only if*

$$\left(|A|, -\sum_{v \in A} p_v\right) \quad \textit{is lexicographically greater than} \quad \left(|C|, -\sum_{v \in C} p_v\right).$$

Proof. If $|A| > |C|$, then

$$W(A) - W(C) = B(|A| - |C|) - \left(\sum_A p - \sum_C p\right) \geq B - 1000|A| \geq B - 100000 = 1 > 0.$$

If $|A| = |C|$, then $W(A) - W(C) = \sum_C p - \sum_A p$, so a larger score means a smaller total price. □

Thus maximizing the single integer W is exactly equivalent to the required lexicographic objective. The largest possible score is at most $100B = 10{,}000{,}100$, which fits safely in a signed 32-bit integer. The score is additive over disjoint components and over selected vertices, so the same encoding can be used by component decomposition and tree DP.

2.2 Branch and Bound

A search state consists of the selected set S, its score s, and the candidate set C of fruits that are compatible with every vertex of S. For a pivot $v \in C$, every feasible completion belongs to exactly one of two cases:

$$\begin{aligned} &\text{take } v: \quad (S \cup \{v\},\ s + w_v,\ C \setminus (\{v\} \cup N(v))), \\ &\text{skip } v: \quad (S,\ s,\ C \setminus \{v\}). \end{aligned}$$

The take branch is searched first. This is a heuristic for finding a good incumbent early; it is not required for correctness.

The search is pruned with an upper bound $\mathrm{UB}(C)$ on the score that can still be added from C. If $s + \mathrm{UB}(C) \leq W_{\text{best}}$, the state cannot strictly improve the incumbent and is discarded. Equality is safe because the encoded score already includes the price tie-break and any optimal set is acceptable.

The pivot is chosen dynamically as a vertex with the largest number of neighbors still in C. Its take branch removes the pivot and all its remaining neighbors, which often reduces the search tree. Its skip branch removes only the pivot, so the rule is a heuristic rather than a guarantee. A static degree order is cheaper per node but becomes less informative as candidates disappear; the final program uses the dynamic rule.

2.3 Matching Upper Bound

Summing all candidate weights is a valid but loose upper bound. A tighter bound is obtained from a greedy matching Q in the induced conflict graph $G[C]$. Let U be the unmatched candidates. Then

$$\mathrm{UB}(C) = \sum_{(u,v)\in Q} \max(w_u, w_v) + \sum_{u\in U} w_u.$$

Lemma 2. $\mathrm{UB}(C)$ *is an upper bound on the score of any independent subset of* C*.*

Proof. The pairs of Q are disjoint. An independent set can contain at most one endpoint of each matched pair, so the contribution of a pair is at most the larger of its two weights. An unmatched vertex contributes at most its own weight. Conflicts between different pairs are ignored, which can only make the bound larger. Hence the sum is an upper bound. □

No maximum matching is required for correctness. The implementation builds the matching greedily, scanning vertices in decreasing initial degree order. A different order changes the tightness of the bound but not its validity.

2.4 Component Decomposition

If the conflict graph has connected components $G_1, \ldots, G_k$, then there are no edges between different components, and

$$\mathrm{OPT}(G) = \sum_{i=1}^{k} \mathrm{OPT}(G_i)$$

for the additive score. The program solves each original component separately and combines the selected masks. This avoids exploring the Cartesian product of independent local choices, which is especially valuable when several components have many tied local optima.

The program does not re-split arbitrary cyclic candidate graphs at every recursive node. However, the forest routine described below can solve several residual trees together inside one component.

2.5 Tree DP

A connected simple graph with n vertices and $n-1$ edges is a tree. For a rooted tree, let $T(v)$ be the best score of the subtree rooted at v when v is selected, and $F(v)$ the best score when v is excluded. Then

$$T(v) = w_v + \sum_{u \text{ child of } v} F(u), \qquad F(v) = \sum_{u \text{ child of } v} \max\{T(u), F(u)\}.$$

The values are computed in reverse traversal order and a choice is reconstructed from the root downward. A selected parent forces every child to be excluded; otherwise the better child state is chosen. The optimum of the tree is $\max\{T(\mathrm{root}), F(\mathrm{root})\}$.

This DP is exact by induction on the height of the subtree. The final program applies it to every original component that is a tree, so paths, stars, and trees are solved without branching.

2.6 Residual Forest DP

Branching removes vertices, so a cyclic component can become a forest. During the search, the program tests the current candidate graph $G[C]$. A traversal checks acyclicity and establishes parent pointers. If a cycle remains, ordinary search continues. If $G[C]$ is a forest, the same take/skip recurrence solves every residual tree, the resulting score is added to s, and that search branch ends immediately.

This substitution is exact. Every residual candidate is compatible with S, there are no edges between different residual trees, and the DP is exact on each tree. The algorithm never approximates a cyclic graph by a tree; it explicitly verifies acyclicity before using the recurrence.

2.7 When to Check: Versions 7 and 8

A forest test costs $O(M+E)$ even when it fails. Testing every search node can waste traversal work while many cycles remain; testing only very late can miss an inexpensive DP completion. Two schedules were developed:

- Version 7 uses fixed thresholds. When the original component has cycle rank $r = e - n + 1 < 3$, it first checks after at least five candidates have disappeared; when $r \geq 3$, it first checks only when at most 15 candidates remain. After a failed check it waits for five more disappearances along that branch.
- Version 8 sets the first threshold from the component's cycle rank:

$$q = \min\{n, \lceil r/2 \rceil\},$$

  and performs the first test after at least q candidates have disappeared. After a failed test, a descendant is tested only when its incoming branch removed at least two candidates. A take branch usually removes a pivot and several neighbors; a skip branch removes one vertex and therefore does not immediately trigger another check.

Versions 7 and 8 use the *same* forest-DP substitution and the same correctness argument. They differ only in when the test is attempted. In other words, v8 is a parameter choice inside v7's mechanism, not a new algorithm.

The schedule is heuristic. The cycle rank counts independent cycles, not the number of vertices that must be deleted: removing one hub can destroy many cycles, while deleting unrelated leaves may destroy none. The rank is computed once per original component and is not recomputed at every state. Delaying or omitting a test cannot lose an optimum; it only leaves more work to exact branch and bound.

A concrete limitation of the formula is that $q = n$ when $r \geq 2n-1$. Then the first test is deferred until $C = \varnothing$, so a dense high-rank component gets no early forest detection at all. For example, a 20-vertex component with 100 edges has $r = 81$ and $q = 20$. This is one reason the schedule should be read as a heuristic parameter, not an optimal rule. On some graphs v8 can be slower than v7, because the extra checks can cost more than the search they save. Correctness is unaffected, because a delayed test only leaves the work to the exact search.

2.8 Final Algorithm

Algorithm 1 shows the final search for one cyclic component. The component size n and the first threshold q are fixed before the call. Isolated fruits are added to the answer

directly and are not part of C.

Algorithm 1 Structural branch and bound (v8, one cyclic component)

procedure SEARCH($C, s, S, \mathit{started}, \Delta$)
  if $s + \mathrm{UB}(C) \le W_{\mathrm{best}}$ **then**
    return
  end if
  if $(\neg \mathit{started} \wedge n - |C| \ge q) \vee (\mathit{started} \wedge \Delta \ge 2)$ **then**
    if $G[C]$ is a forest **then**
      Solve all residual trees by DP and update the incumbent
      return
    end if
    $\mathit{started} \leftarrow$ true
  end if
  if $C = \varnothing$ **then**
    Update incumbent with (s, S); **return**
  end if
  $v \leftarrow$ maximum-degree vertex in $G[C]$
  $C' \leftarrow C \setminus (\{v\} \cup N(v))$
  SEARCH($C', s + w_v, S \cup \{v\}, \mathit{started}, |C| - |C'|$)
  SEARCH($C \setminus \{v\}, s, S, \mathit{started}, 1$)
end procedure

2.9 Correctness

Theorem 1. *For every valid PTA input, the program outputs a feasible set of maximum cardinality; among all such sets it outputs one of minimum total price.*

Proof. By Lemma 1, maximizing W is equivalent to the required lexicographic objective. For a pivot v, every independent set either contains v or does not; the take and skip branches therefore partition all feasible completions. The matching bound is a valid upper bound by Lemma 2, so a state is discarded only when it cannot strictly improve the incumbent. Component decomposition is valid because the score is additive and there are no cross-component edges. The tree and forest DPs are exact by induction on subtree height and are invoked only after the acyclicity check. Therefore the search never discards an optimal set, and the reconstructed mask is feasible and optimal. The final sorting by ID only changes the output order, not the selected set. □

2.10 Complexity

Let K be the number of visited search states, M the basket size, and E the number of represented conflict edges. The matching bound, the dynamic pivot, and a triggered forest check each cost $O(M + E)$ in a conservative bound. The total time is therefore $O(K(M + E))$, with $K = O(2^M)$ in the worst case.

The fixed-size adjacency arrays use $O(M^2)$ space and the recursion $O(M)$ stack space; with $M \le 100$ the dense array is small and two 64-bit words cover all candidates. The worst case remains exponential, but the pruning and structural routines make the PTA limits comfortable.

Chapter 3. Testing Results

3.1 Test Cases and Input Data

The cases cover the PTA sample, boundary sizes, the main graph shapes, the lexicographic objective, outside-basket warnings, and output formatting. Table 3.1 lists them. The two large cases are constructed as follows:

- **K14**: all $\binom{14}{2} = 91$ pairs are warnings; prices are 1, 2, ..., 14. The optimum is the single cheapest fruit.
- **Path100**: warnings form the path $001-002-\cdots-099-100$, plus the outside warning `001 999`; odd IDs have price 1 and even IDs have price 100. The unique optimum is the 50 odd IDs, with total price 50.

The other cases are small enough to be read directly from their descriptions.

Table 3.1: Test cases and results. "same" means the actual output was byte-for-byte identical to the expected output.

No.	Purpose	N/M	Expected (count / IDs / price)	Actual	Result
1	PTA sample; isolated fruits 019/020	16/20	`12 / 002 004 006 008 009 014 015 016 017 018 019 020 / 239`	same	Pass
2	Single fruit; outside tip	1/1	`1 / 001 / 5`	same	Pass
3	All isolated; every fruit selected	3/5	`5 / 001 002 003 004 005 / 150`	same	Pass
4	Complete K_4; count 1, cheapest	6/4	`1 / 001 / 10`	same	Pass
5	Path with cheap middle; count first	2/3	`2 / 001 003 / 200`	same	Pass
6	Path; same count, lower price	3/4	`2 / 001 004 / 2`	same	Pass
7	Two disjoint edges; components	2/4	`2 / 001 003 / 4`	same	Pass
8	4-cycle; same count, lower price	4/4	`2 / 001 003 / 4`	same	Pass
9	Star; leaves beat the centre	3/4	`3 / 002 003 004 / 60`	same	Pass
10	Leading zeros 010 and 100	1/2	`2 / 010 100 / 12`	same	Pass
11	K_{14}; 91 tips, dense	91/14	`1 / 001 / 1`	same	Pass
12	Path100; max $N = M = 100$	100/100	`50 / 001 003 ... 097 099 / 50`	same	Pass

3.2 Measurement Method

The tests were run on Windows with gcc 15.2.0 (MinGW-w64), using `gcc -O2 -std=c99`. Each case was run as a separate process with the test input on standard input. The output was compared with the expected output, and a checker verified:

- the first line equals the number of IDs on the second line;
- the IDs are sorted, unique, and present in the basket;
- no forbidden pair is selected;

- the third line equals the sum of the selected prices;
- the count and price equal the expected optimum.

3.3 Sample Run

The exact output for the PTA sample is shown below. It matches the required format: three lines, one space between IDs, no trailing space.

```
12
002 004 006 008 009 014 015 016 017 018 019 020
239
```

3.4 Additional Automated Tests

Two repository scripts provide broader coverage:

- `test_random.py` generates 225 random small cases, computes the optimum by enumerating all subsets, and checks every version. The final program and all earlier versions pass.
- `test_special.py` checks five special cases: objective priority, multiple optima, dirty input (outside tips and a duplicate tip), multiple components, and a maximum-size case with 50 disjoint edges. All versions pass. The multiple-optima case is stricter than the PTA guarantee, which says the answer is unique; it verifies that any optimal set is accepted by the checker.

In total, the final program passed the 12 hand-written cases, the 225 random cases, and the 5 special cases. The sample output and the formatting checks also passed.

3.5 Performance by Optimization

3.5.1 Benchmark Method

All comparisons use the same full suite of 2,460 labeled inputs, including different graph families, sizes, prices, and shuffled fruit IDs. Programs were built with `gcc -O2 -std=c99 -DBENCH`; `QueryPerformanceCounter` measured the solve phase inside each program. The runs below used one timing per input and a three-second timeout. A timeout is recorded as 3,000 ms, so any sum containing timeouts is a lower bound on actual completion time. Completed answers were checked against the oracle. Individual sub-millisecond measurements are noisy; the family medians and timeout counts are more useful than small differences between individual cases.

The component-decomposition experiment uses an earlier run of the same suite with a five-second timeout and five repeats for fast cases. Its numbers are compared only within that run. The branch run is recorded in `bench/results_branch_235.csv`. The earlier run is in `bench/results_full.csv`. The structural run is in `bench/results_v5_v8.csv`.

3.5.2 Branch Selection: v2, v3, and v5

Version 3 takes the lowest-index remaining fruit; version 5 sorts fruits once by their initial conflict degree and searches in that order; version 2 selects a maximum-degree fruit anew from the current candidate graph at each node. The same-run results are in Table 3.2.

Table 3.2: Branch-strategy run, 2,460 inputs per version (ms unless stated).

Version	Median	p95	Capped sum (s)	Timeouts
v2: dynamic degree	0.0650	25.9764	16.712	0
v3: fixed index	0.0416	243.8242	151.284	9
v5: static degree order	0.0452	84.1368	62.910	4

All three versions decompose connected components. The v2/v5 pair therefore compares dynamic and static degree ordering with decomposition held constant; v3 uses the fixed input-index order. On 100-fruit banded graphs, the medians are 87.398 ms for v2, 544.483 ms for v3, and 373.276 ms for v5. On the 100-fruit components family, v5 is slightly faster than v2 (0.0330 versus 0.0549 ms). Across the full suite, v5 has a lower overall median than v2, but a higher p95 and four timeouts, while v2 has none. Static ordering can work well on simpler or disconnected graphs; as candidates disappear in a large connected graph, initial degrees become less informative and dynamic selection follows the remaining conflicts more closely.

3.5.3 Connected-Component Decomposition: v1–v4

The first four versions form two controlled pairs: v1 and v3 both use a fixed index, while v4 and v2 both use a dynamic maximum-degree pivot. Within each pair, only the latter version decomposes connected components. Table 3.3 uses the earlier five-second run.

Table 3.3: Effect of decomposition in the original v1–v4 run.

Version	Decompose?	Components100 (ms)	Path100 (ms)	Timeouts/2460
v1: fixed	no	155.275	362.565	68
v3: fixed	yes	0.046	358.084	10
v4: dynamic	no	76.470	149.884	21
v2: dynamic	yes	0.083	150.025	0

The components family consists of disconnected subgraphs. Solving each one separately removes combinations of independent choices, giving a large gain in both controlled pairs. A path is already one connected component, so its paired times remain close.

3.5.4 Original Trees: v2 and v6

Version 6 adds tree DP to v2. It recognizes components with $e = n - 1$ and solves them by the take/skip recurrence before any branching. In the updated v2/v5/v6/v7/v8 run, the median for 100-fruit paths falls from 80.479 to 0.0046 ms, and for 100-fruit trees from 4.521 to 0.0071 ms. A 100-fruit cycle is still cyclic at the start: v2 takes 69.597 ms and v6 takes 67.204 ms. This comparison locates the gain specifically in original tree components.

3.5.5 Cycles That Become Forests: v6 and v7

Version 7 tests whether a cyclic component's *remaining candidates* have become a forest during backtracking. If so, forest DP finishes that branch exactly. The v7 trigger is deliberately delayed: for initial cycle rank $r = e - n + 1 < 3$, the first test follows five

disappeared candidates; for $r \geq 3$, it waits until at most 15 remain. After a failed test, another five candidates must disappear before the next test on that branch.

In Table 3.4, v6 and v7 use the same pivot, bound, component decomposition, and original-tree DP. Thus the cycle and banded gains measure the residual-forest mechanism. The original 100-fruit cycles take 67.204 ms in v6 and 0.0423 ms in v7; banded graphs take 86.048 and 0.0381 ms.

3.5.6 Choosing the Check Time: v7 and v8

For a connected component, $r = e - n + 1$ is the minimum number of *edges* that must be removed to leave a forest. Deleting one vertex can remove several incident edges, so v8 uses $q = \min\{n, \lceil r/2 \rceil\}$ disappeared candidates as a heuristic first-check threshold. This is an estimate for when a forest check may become worthwhile, not a bound on the number of vertex deletions needed. After an unsuccessful check, v8 checks a descendant only when the incoming branch removes at least two candidates; taking a fruit often removes its neighbors too, whereas skipping it removes only one.

Table 3.4: Structural run: same 2,460 inputs, one timing each (ms).

Version	p95	Capped sum (s)	Path100	Cycle100	Banded100
v2	25.1449	16.721	80.4787	69.5972	101.7803
v5	83.3704	62.763	196.4078	236.3376	427.0492
v6	10.5094	11.041	0.0046	67.2045	86.0485
v7	2.2941	1.612	0.0047	0.0423	0.0381
v8	0.7945	0.773	0.0047	0.0147	0.0141

Version 8 changes only the forest-check schedule, and its cumulative solve time falls from v7's 1.612 s to 0.773 s. The gain is graph-dependent: 100-fruit core-periphery graphs rise from 0.0546 to 0.0967 ms because extra checks can cost more than they save. Four three-second timeouts occurred, all in v5; v2 and v6–v8 completed every case. The schedule changes running time only; both versions use the same exact DP after a successful forest test.

Chapter 4. Analysis and Comments

4.1 Why the Algorithm Is Exact

The proof has four independent parts. The score encoding turns the two objectives into one integer. The take/skip branches partition all independent sets. The matching bound is a valid upper bound, so pruning never removes a state that could improve the incumbent. Component decomposition and tree or forest DP are exact on their subproblems. The forest check is separated from the DP: the check may be delayed, but when it succeeds the DP is exact, and when it fails the ordinary search continues.

This separation is the main design idea. A heuristic trigger is allowed to affect speed only; it cannot affect the answer.

4.2 Why the Optimizations Matter

The matching bound discards score combinations that cannot beat the incumbent. Component decomposition avoids the Cartesian product of independent local choices. The dynamic pivot removes a high-degree vertex and many neighbors in the take branch. Tree DP replaces enumeration on a path, star, or tree, and residual-forest DP extends the same idea to the acyclic remainder created by branching.

On the 100-vertex path and cycle, the final program solves the problem in a few microseconds of internal time, while the baseline branches for milliseconds. The improvement is not a constant-factor tweak; it changes the work required for a whole tractable subproblem.

4.3 PTA-Specific Pitfalls

1. **Outside warnings (defensive).** A warning whose endpoint is not in the basket is ignored. The PTA sample does not exercise this rule (the lines `020 99` and `019 99` are fruit lines), but the special tests do. Ignoring such a warning is safe, because a fruit that is not in the basket cannot be selected.

2. **Count before price.** A cheap fruit may block two expensive fruits. The score $B - p_v$ makes the count dominant; do not sort by price or use a price-only greedy rule.

3. **Unique answer.** The problem guarantees a unique optimum, but the program still implements the price tie-break. It does not need to enumerate all optimal sets.

4. **Three-digit IDs.** Print with `%03d`; sort numerically, not lexicographically. The leading-zero case `010/100` checks this.

5. **Output format.** Exactly three lines; IDs in increasing order; exactly one space between IDs; no trailing space. The count must equal the number of IDs, and the price must equal their sum.

6. **Maximum size.** $M = 100$ fits in two 64-bit masks. The maximum total price is 100000, so $B = 100001$ is large enough; the maximum score fits in a signed 32-bit integer.

7. **Isolated fruits.** A fruit with no warning is always selected, because its weight is

positive. The program adds isolated fruits directly and removes them from the search.

8. **Input assumptions.** The statement says there are no duplicate tips. The program assumes a simple graph; it does not specially handle self-loops, which the statement does not define.

4.4 Limitations

The worst case is still exponential: maximum independent set is NP-hard, and an adversarial graph with $M = 100$ can make branch and bound explore many states. The program is exact, so it will eventually finish, but a malicious test could be slow. The forest-detection schedule is a heuristic; its parameters were chosen to work well on the tests, and v8 can be slower than v7 on some graphs, especially when branching is already cheap. This is a performance trade-off, not a correctness trade-off. The v7 schedule is a simpler alternative if robustness across graphs is more important than aggregate speed.

4.5 Lessons Learned

- Start from a correct baseline and add one optimization at a time; test after each change.
- Use brute force on small random cases to validate the objective and the tie-break, not just the sample.
- Keep the exact structural solver separate from the heuristic that decides when to call it. This makes the correctness argument independent of the tuning.
- Output formatting is part of the PTA contract: one extra character can fail an otherwise correct solution.
- Performance measurements are useful, but a single observation on one machine should not be over-interpreted.

4.6 Summary

The final program solves the PTA Safe Fruit problem exactly. It encodes the count-first, price-second objective as one integer, prunes with a matching bound, decomposes connected components, and uses tree and residual-forest DP to avoid branching on acyclic structure. It handles the PTA-specific cases: defensive handling of outside warnings, leading-zero IDs, maximum size, and the exact output format.

The program passed the sample, 12 designed cases, 225 random brute-force cases, and 5 special cases. Versions 7 and 8 share the same forest-DP mechanism; v8 only changes when the forest check is attempted. The v8 schedule is faster in aggregate on the tested cases, but it is a parameter choice, not a universally better algorithm, and some graphs can regress. For the PTA assignment, the final program is exact, well tested, and comfortably fast for $M \leq 100$.

References

[1] *Safe Fruit*, PTA programming assignment, Zhejiang University.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 4th edition. MIT Press, 2022.

Chapter A. Source Location

The final program and the development versions are available at

https://github.com/blxrua/ads_project3_ads (main branch).

The report source, compiled PDF, and the 12 hand-written test inputs are included in the repository root and the `tests/` directory. The relevant files are:

- `report_safe_fruit.tex` and `report_safe_fruit.pdf`: this report;
- `v8_cycle_tree_dp.c`: the final program;
- `v1_steps123.c` through `v7_cycle_tree_dp.c`: development history used for regression testing;
- `test_random.py`: 225 random cases checked against brute force;
- `test_special.py`: five special cases, including dirty input and the maximum-size case;
- `sample.in`: the PTA sample input.

The final program was compiled with

```
gcc -O2 -std=c99 -o safe_fruit v8_cycle_tree_dp.c
```

and run with the input on standard input. The optional timing build uses `-DBENCH`; it is not needed for the PTA submission.